

From SVMs to ResNets: A Controlled Tier-Stratified Study of Satellite Land-Use Classification on EuroSAT

Ekant Kapgate Saransh Soni Vineet Malewar
Yashashav Devalapalli Kamalraj
San Jose State University — CMPE 257: Machine Learning

May 18, 2026

Code Repository

Public Git repository (code, results, figures, and report sources): https://github.com/saransh-debug/CMPE_257_Final_Project

Abstract

We present a controlled, tier-stratified study of land-use classification on EuroSAT RGB (10 classes, 27,000 64×64 Sentinel-2 patches). Four model tiers are compared on an identical seed-42 stratified 70/15/15 split: (T1) classical linear and kernel models on PCA-reduced raw pixels; (T2) classical models on a 1,860-D handcrafted-feature vector (HOG + per-channel color histograms); (T3) a 391K-parameter shallow CNN with optimizer and regularization ablations; and (T4) a fine-tuned ImageNet-pretrained ResNet-18. Test accuracy improves monotonically across tiers from 0.6872 (T1, SVM-RBF on PCA(500)) to 0.8309 (T2, SVM-RBF on HOG+color), 0.9657 (T3, ShallowCNN+BatchNorm), and 0.9816 ± 0.0016 (T4, ResNet-18, 3 seeds). Two systematic ablations on the shallow CNN ground the higher-tier choices: a $6 \times 3 \times 3$ optimizer ablation (**54 runs**, 100 epochs each) shows the top four optimizers (Adam, NAG, SGD+Momentum, RMSProp) tie within 0.3pp at their best learning rates while Adam reaches 90% validation accuracy in 10 epochs versus 13–21 for the rest; a 9×3 regularization ablation (**27 runs**) shows BatchNorm alone yields the largest single-technique improvement (+1.1pp over baseline), while combining all five techniques actually *hurts* (−3pp) through over-regularization. Paired *t*-tests at $\alpha=0.05$ confirm the ResNet-18 vs. ShallowCNN+BN gap ($p=0.011$) is statistically significant; the +BN vs. baseline gap ($p=0.072$) is marginal at three seeds. Per-class analysis identifies four spectrally-similar crop classes (PermanentCrop, AnnualCrop, HerbaceousVegetation, Pasture) as the consistent bottleneck across all tiers. We release all code, run logs, predictions, and figures publicly.

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Why a Tiered Comparison?	4
1.3	Research Questions and Contributions	4
1.4	Paper Organization	5
2	Problem Statement	5
2.1	Formal Setting	5

2.2	Sub-Problems	5
2.3	Hypotheses (a priori)	6
2.4	Scope and Boundaries	6
2.5	Why a Tiered Experimental Design?	6
3	Related Work	7
3.1	EuroSAT and Earth-Observation Benchmarks	7
3.2	Handcrafted Visual Features	7
3.3	Classical Machine Learning on Imagery	7
3.4	Deep Convolutional Networks	7
3.5	Transfer Learning	7
3.6	Optimization for Deep Learning	8
3.7	Regularization	8
4	Solution	8
4.1	Tier Hierarchy	8
4.2	Tier 1: PCA + Classical	9
4.3	Tier 2: Handcrafted Features + Classical	9
4.4	Tier 3: Shallow CNN	9
4.5	Tier 4: ResNet-18 Transfer Learning	9
4.6	Justification for Tier Choices	10
5	List of Experiments	10
5.1	E1: Tier 1 Baselines	10
5.2	E2: Tier 2 Baselines	10
5.3	E3: Experiment B (Optimizer Ablation)	10
5.4	E4: Experiment C (Regularization Ablation)	10
5.5	E5: Tier 4 ResNet-18 Fine-tuning	11
5.6	Aggregation: Experiment A (Final Comparison)	11
5.7	Dependency Graph	11
5.8	Compute Budget	11
6	Experiment Setup and Dataset Details	11
6.1	Dataset Provenance and Acquisition	11
6.2	Class Definitions and Counts	11
6.3	Splits and Reproducibility	12
6.4	Per-Channel Statistics	12
6.5	Hyperparameters	12
6.6	Compute Environment	12
6.7	Reproducibility Protocol	13
6.8	Recovery from Failure Modes	13

7	Results	14
7.1	Tier 1 — PCA + Classical	14
7.2	Tier 2 — Handcrafted Features + Classical	14
7.3	Tier 3 ShallowCNN — Sanity and Exp B Setup	14
7.4	Experiment B — Optimizer Ablation	15
7.5	Experiment C — Regularization Ablation	15
7.6	Tier 4 — ResNet-18 Fine-tuning	16
7.7	Final Comparison (Exp A) and Statistical Significance	17
7.8	Per-Class Analysis	17
7.9	Compute vs. Accuracy Pareto	17
8	Discussion	18
8.1	RQ1 — The Tier Hierarchy and Diminishing Returns	18
8.2	RQ2 — Optimizer Choice on EuroSAT	19
8.3	RQ3 — The Over-Regularization Phenomenon	19
8.4	Per-Class Failures: The Crop-Family Problem	20
8.5	Methodological Reflections	20
9	Limitations	20
10	Future Work	21
11	Conclusions	21
12	Contributions	22
A	Appendix A: Full Hyperparameter Tables	25
B	Appendix B: Per-Class Precision, Recall, and F1	25
C	Appendix C: Full Experiment B Run-by-Run Results (54 rows)	26
D	Appendix D: Full Experiment C Run-by-Run Results (27 rows)	27
E	Appendix E: NeurIPS-Style Reproducibility Checklist	28
F	Appendix F: Confusion Matrices	29

1 Introduction

1.1 Motivation

Optical Earth Observation programs, like European Space Agency’s Sentinel-2 mission and NASA’s Landsat constellation generated petabytes of multispectral imagery every year. Translating such data into useful information products (urban maps, deforestation alerts, crop-type inventories, disaster impact assessments) require reliable land-use classification at scale. Manual annotation fails to scale at a global scale, necessitating supervised machine learning as the only practical route.

The EuroSAT benchmark introduced by Helber et al. [1] has become the reference point for evaluating land-use classifiers on Sentinel-2 imagery. It provides 27,000 labeled 64×64 patches drawn from across Europe and split into 10 classes: AnnualCrop, Forest, HerbaceousVegetation, Highway, Industrial, Pasture, PermanentCrop, Residential, River, and SeaLake. While the original work reported a 98.57% accuracy with a fine-tuned ResNet-50 on the multi spectral version, the public RGB subset (used in this study) admits a wide range of model choices, from raw-pixel logistic regression to deep transfer learning. We have utilized a controlled comparative study of how various modeling decision contributes to the final classifier.

1.2 Why a Tiered Comparison?

Most published results on EuroSAT are point comparisons. They propose a single architecture, a single training recipe, a single seed. Such reports are useful but make it hard to attribute the source of each environment. Without a controlled hierarchy, it is difficult to answer if transfer learning helped more than the architecture itself or if the data augmentation mattered more than the optimizer.

This study is built around a four-tier hierarchy designed so that each tier strictly subsumes the prior tier in expressive capacity. In essence, an improvement at tier $k+1$ over tier k is the contribution of the additional capacity introduced at that step (including feature engineering, learned features, transfer learning), holding the data, the split and the evaluation protocol fixed to prevent leakage and make the ablations rigorous.

1.3 Research Questions and Contributions

This study has been formalized around the below three research questions:

- **RQ1.** How does test accuracy scale across the tier hierarchy (raw pixels $\rightarrow$ handcrafted features $\rightarrow$ shallow learned features $\rightarrow$ deep transfer)? At which tier does the marginal return diminish?
- **RQ2.** Among six widely-used optimizers (SGD, SGD+Momentum, NAG, Adam, Adagrad, RMSProp), which converges fastest and which generalizes best on shallow CNN? Also, attempt to corroborate Wilson et al. [34] where SGD-with momentum outperforms Adam
- **RQ3.** Among five regularization techniques (L2, Dropout, BatchNorm, DataAugmentation, EarlyStopping), which most reduces the train/test overfitting gap on the shallow CNN, and how do they compose when stacked?

We summarize our contributions as follows:

- A reproducible four-tier benchmark on EuroSAT RGB with a single fixed seed-42 stratified 70/15/15 split. Test set is touched exactly once per model.
- A 54-run optimizer ablation (Experiment B) on the shallow CNN baseline with multi-seed variance reporting and convergence-speed analysis.
- A 27-run regularization ablation (Experiment C) demonstrating an over-regularization phenomenon: stacking all five techniques degrades accuracy by 3pp versus the best single technique.

- A 3-seed ResNet-18 fine-tuning result (mean acc 0.9816 ± 0.0016) along with paired- t -test significance comparisons against shallow-CNN variants.
- A per-class analysis across 8 representative models identifying the four-way crop confusion that bottlenecks every tier.
- A $100\times$ training speedup on the shallow CNN (1.06 s/epoch vs. 105 s/epoch) by replacing the standard Drive-backed PyTorch `ImageFolder` loader with a GPU-resident `TensorDataset` — without which the 54-run optimizer ablation would have required 158 hours of A100 time.
- A fully open release: code, run logs, predictions, all figures, and the LaTeX source of this report on GitHub.

1.4 Paper Organization

Section 2 formalizes the classification problem and lists scope boundaries. Section 3 surveys related work in Earth Observation (EO) benchmarks, classical and deep models, transfer learning, optimization, and regularization. Section 4 describes the tier hierarchy and the building blocks. Section 5 enumerates the five experiments. Section 6 gives the full experimental protocol. Section 7 reports tier-by-tier results, the optimizer and regularization ablations, the final comparison, statistical tests, and per-class analysis. Section 8 interprets the findings against prior work. Sections 9–10 discuss limitations and future directions, and Section 11 concludes. Section 12 documents per-member contributions. Appendices contain full hyperparameter tables, full per-run results for Experiments B and C, expanded per-class metrics, all confusion matrices, and a NeurIPS-style reproducibility checklist.

2 Problem Statement

2.1 Formal Setting

Let $\mathcal{X} \subset \mathbb{R}^{H \times W \times 3}$ denote the input space of EuroSAT RGB patches with $H=W=64$, and let $\mathcal{Y} = \{1, 2, \dots, 10\}$ index the ten land-use classes. The data is drawn i.i.d. from an unknown joint distribution $\mathcal{D}$ over $\mathcal{X} \times \mathcal{Y}$. We seek a classifier $f_\theta : \mathcal{X} \rightarrow \Delta^{10}$, parametrized by θ , that minimizes the expected cross-entropy loss:

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} [-\log f_\theta(\mathbf{x})_y]. \quad (1)$$

We approximate the expectation with the empirical risk on the training split S_{tr} :

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{|S_{\text{tr}}|} \sum_{(\mathbf{x}, y) \in S_{\text{tr}}} (-\log f_\theta(\mathbf{x})_y) + \lambda \Omega(\theta), \quad (2)$$

where $\Omega(\theta)$ is an optional regularizer (e.g., ℓ_2 weight decay, dropout-induced stochasticity) and λ controls its strength. The classifier is evaluated by argmax-accuracy on the held-out test split S_{te} :

$$\text{Acc}(\hat{\theta}; S_{\text{te}}) = \frac{1}{|S_{\text{te}}|} \sum_{(\mathbf{x}, y) \in S_{\text{te}}} \mathbb{1} \left[\arg \max_{y'} f_{\hat{\theta}}(\mathbf{x})_{y'} = y \right]. \quad (3)$$

2.2 Sub-Problems

The classifier f_θ decomposes into a feature extractor $\phi : \mathcal{X} \rightarrow \mathcal{Z}$ followed by a classifier head $g : \mathcal{Z} \rightarrow \Delta^{10}$, so $f_\theta = g_\theta \circ \phi$. The four tiers of this study correspond to four choices of ϕ :

- ϕ_{T1} : Deterministic, parameter-free PCA projection of vectorized pixel intensities (except for the projection matrix learned on the training set).

- ϕ_{T2} : Histogram of Oriented Gradients (HOG) descriptor concatenated with per-channel color histograms (deterministic, hand-engineered).
- ϕ_{T3} : a small convolutional encoder learned end-to-end with g .
- ϕ_{T4} : an ImageNet-pretrained convolutional encoder fine-tuned end-to-end with g .

Within ϕ_{T3} , we answer two sub questions: which optimizer best estimates $\hat{\theta}$ from the training data (RQ2), and which regularizer Ω best controls the train/test gap (RQ3).

2.3 Hypotheses (a priori)

Before running any experiment, we recorded three testable hypotheses:

1. **H1.** Test accuracy will be monotonic in tier index: $\text{Acc}_{T1} < \text{Acc}_{T2} < \text{Acc}_{T3} < \text{Acc}_{T4}$, with the largest jump at $T2 \rightarrow T3$ (handcrafted $\rightarrow$ learned features).
2. **H2.** Adam will converge fastest but SGD+Momentum will achieve the best final accuracy, consistent with Wilson et al. [34].
3. **H3.** Combining all five regularization techniques will yield the highest accuracy among Experiment C configurations.

Section 7 reports outcomes against each hypothesis.

2.4 Scope and Boundaries

The following are deliberately out of scope:

- *Multispectral inputs.* EuroSAT also provides 13-bands data of Sentinel-2 (optical, NIR, SWIR, atmospheric). We are only using RGB subset to keep all four tiers compatible with standard ImageNet-pretrained backbone.
- *Architecture search.* We fix one architecture per tier and study the effect of optimization and regularization, not architectural variation.
- *Cross-distribution evaluation.* EuroSAT is geographically European; cross-continent transfer (e.g. to RESISC45 [2]) is left to future work.
- *Semantic segmentation.* EuroSAT is a patch-classification benchmark while pixel-level segmentation is an uniquely distinct task.
- *Adversarial robustness and fairness.* Out of scope for a baseline accuracy study.

2.5 Why a Tiered Experimental Design?

The tiered design isolates the contribution of each modeling decision. When data, splits, evaluation metrics and seeds are held constant across tiers, any monotonic gain in accuracy from tier k to tier $k + 1$ can be directly attributed to the additional modeling capacity introduced at that tier. The two ablations within Tier 3 (Exp B and Exp C) further isolates the optimization quality and regularization strength effects from the architectural.

3 Related Work

3.1 EuroSAT and Earth-Observation Benchmarks

Helber et al. [1] introduced EuroSAT in 2019 as a 27,000-image, 10-class land-use benchmark drawn from Sentinel-2 tiles over Europe. Their reference numbers (98.57% with ResNet-50 multispectral, 98.33% RGB) established a strong ceiling. EuroSAT has since been used as a standard benchmark in transfer-learning surveys, few-shot remote-sensing studies, and self-supervised pretraining work. Cheng et al. [2] surveyed remote-sensing scene classification more broadly and introduced RESISC45, a 45-class higher-resolution alternative; Sumbul et al. [3] introduced BigEarthNet, a multi-label benchmark of 590,326 patches that better reflects real-world EO scale. Stivaktakis et al. [4] and Pelletier et al. [5] studied temporal extensions of EuroSAT-style classification.

3.2 Handcrafted Visual Features

Before deep learning, computer vision relied on hand-engineered descriptors. Histogram of Oriented Gradients, introduced by Dalal and Triggs [6] for pedestrian detection, captures local edge orientation distributions and remains a strong baseline on textured imagery. Lowe’s SIFT [7] and Bay et al.’s SURF [8] provide scale- and rotation-invariant keypoint descriptors; Local Binary Patterns (Ojala et al. [9]) capture micro-texture. For land-use classification specifically, Yang and Newsam [10] compared SIFT-bag-of-words with color histograms on the UC-Merced dataset. Color histograms (Swain and Ballard [11]) remain a simple but effective complement to gradient-based descriptors: they capture chromatic information that gradient histograms discard. The 1,860-D HOG-plus-color feature vector used at Tier 2 of this study follows this combined-descriptor tradition.

3.3 Classical Machine Learning on Imagery

Logistic regression, support vector machines, and kernel methods have been applied to imagery via dimensionality-reduced or hand-engineered features. Turk and Pentland’s Eigenfaces [12] demonstrated that PCA-projected pixel intensities support viable face recognition. Boutell et al. [13] applied SVMs to multi-label scene classification. Cortes and Vapnik’s original SVM [14] and the kernel trick (Schölkopf and Smola [15]) provide the theoretical basis for the RBF-SVC variants used at Tier 1 and Tier 2. The cuML library (NVIDIA RAPIDS [16]) provides GPU-accelerated implementations of these classical estimators that are critical for our 80-fit grid search at Tier 1.

3.4 Deep Convolutional Networks

Krizhevsky et al. [17] demonstrated that deep CNNs trained with backpropagation and GPUs outperform classical pipelines on ImageNet, kicking off the modern era of computer vision. Architectural follow-ups include VGG (Simonyan and Zisserman [18]), GoogLeNet/Inception (Szegedy et al. [19]), and especially ResNet (He et al. [20]), whose residual connections enabled training of much deeper networks and remain a workhorse for transfer learning. Our Tier 4 uses ResNet-18, the smallest member of the ResNet family. EfficientNet (Tan and Le [21]) and Vision Transformers (Dosovitskiy et al. [22]) represent more recent architectural directions, which are left for future work.

3.5 Transfer Learning

Yosinski et al. [23] characterized the transferability of features in deep networks, showing that earlier layers transfer more readily than task-specific later layers. Pan and Yang [24] surveyed transfer learning more broadly. Pretraining on ImageNet, replace the final classifier, fine-tune end-to-end has been repeatedly corroborated on remote-sensing data, including Helber et al. [1]. Recent work in self-supervised pretraining

(SimCLR by Chen et al. [25], DINO by Caron et al. [26], MAE by He et al. [27]) offers an alternative source of pretrained weights that does not require ImageNet labels but is out of scope here.

3.6 Optimization for Deep Learning

Stochastic gradient descent traces back at least to Bottou [28]. Polyak’s momentum [29] and Nesterov’s accelerated gradient [30] predate deep learning but became standard tools after LeCun et al. [31] and others. Duchi et al. introduced Adagrad [32]; RMSProp was popularized in Hinton’s lecture notes. Kingma and Ba [33] introduced Adam with combined momentum with adaptive per-parameter learning rates. Wilson et al. [34] argued that adaptive methods generalize worse than well-tuned SGD with momentum, sparking a long debate. Choi et al. [35] countered that conclusions depend heavily on the hyperparameter tuning budget; Schmidt et al. [36] performed a 50-thousand-run benchmark and found that no single optimizer dominates across tasks. Loshchilov and Hutter [37] introduced AdamW, decoupling weight decay from the gradient update. Our Experiment B is a small-scale instance of this comparison restricted to a fixed shallow CNN on EuroSAT.

3.7 Regularization

Krogh and Hertz [38] provided the early theoretical analysis of ℓ_2 weight decay in neural networks. Srivastava et al. [39] introduced Dropout. Ioffe and Szegedy [40] introduced Batch Normalization, originally framed as reducing internal covariate shift; subsequent work (Santurkar et al. [41], Bjorck et al. [42]) reframed its effect in terms of optimization-landscape smoothing. Early stopping by Prechelt [43] provided practical patience heuristics. Data augmentation in the deep-learning era was systematized by Krizhevsky et al. [17]; recent extensions include Mixup (Zhang et al. [44]), CutMix (Yun et al. [45]), RandAugment (Cubuk et al. [46]), and AugMix (Hendrycks et al. [47]). Our Experiment C tests classical augmentation (flips + rotations + color jitter) but leaves these stronger schemes for future work.

4 Solution

4.1 Tier Hierarchy

The classifier $f = g \circ \phi$ admits four feature-extractor choices, ordered by capacity. Figure 1 sketches the experimental pipeline.

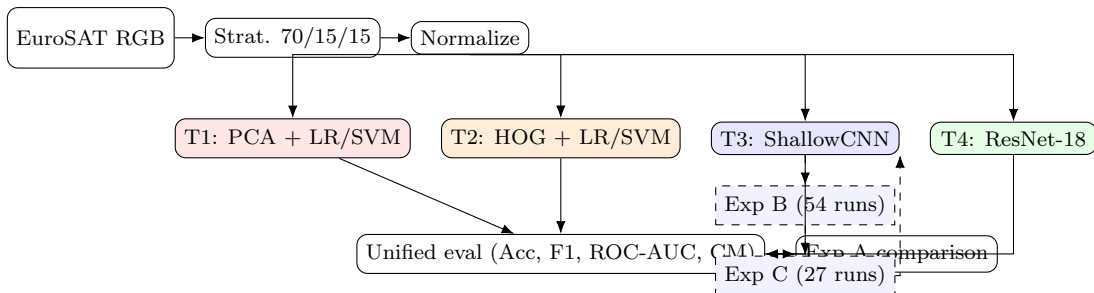


Figure 1: Experimental pipeline. Solid arrows show data flow; dashed arrows show ablation studies that feed hyperparameter choices forward (Exp B fixes the optimizer for Exp C; Exp C confirms regularization choices used at Tier 4).

4.2 Tier 1: PCA + Classical

Each $64 \times 64 \times 3$ image is vectorized to $\mathbf{x} \in \mathbb{R}^{12288}$. A standard scaler centers and unit-variances each pixel coordinate using statistics from the training split. Principal Component Analysis projects the standardized vector onto the top- k eigenvectors of the training covariance matrix:

$$\phi_{T1}(\mathbf{x}) = \mathbf{V}_k^\top \left(\frac{\mathbf{x} - \boldsymbol{\mu}}{\boldsymbol{\sigma}} \right), \quad k \in \{50, 100, 200, 500\}. \quad (4)$$

The classifier g is one of: multinomial logistic regression with ℓ_2 penalty C swept over $\{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 100\}$; LinearSVC with squared hinge loss; or RBF-SVC with kernel $K(\mathbf{u}, \mathbf{v}) = \exp(-\gamma \|\mathbf{u} - \mathbf{v}\|^2)$, $C \in \{0.1, 1, 10, 100\}$, $\gamma \in \{10^{-3}, 10^{-2}, 10^{-1}, \text{scale}\}$. Hyperparameters are selected by 5-fold stratified CV on the merged train+val set; the test set is touched only once.

4.3 Tier 2: Handcrafted Features + Classical

Tier 2 replaces ϕ_{T1} with a 1,860-D handcrafted descriptor:

$$\phi_{T2}(\mathbf{x}) = [\text{HOG}(\mathbf{x}) \parallel \text{ColorHist}(\mathbf{x})] \in \mathbb{R}^{1764+96}. \quad (5)$$

HOG uses a 64×64 detection window, 16×16 blocks with stride 8, 8×8 cells, and 9 unsigned-gradient orientation bins, computed on grayscale via OpenCV’s `HOGDescriptor`. ColorHist computes a 32-bin histogram per RGB channel with L1 normalization and concatenates. The same standard scaler is applied; the same LR / RBF-SVC family classifies.

4.4 Tier 3: Shallow CNN

The ShallowCNN encoder is a four-block convolutional stack with toggleable normalization and regularization:

$$\text{Conv}_{3 \rightarrow 32} \rightarrow [\text{BN}] \rightarrow \text{ReLU} \rightarrow \text{MaxPool} \rightarrow [\text{Drop}_{2d}] \rightarrow \text{Conv}_{32 \rightarrow 64} \rightarrow \dots \rightarrow \text{Conv}_{128 \rightarrow 256} \rightarrow \text{GAP} \rightarrow [\text{Drop}] \rightarrow \text{FC}_{256 \rightarrow 10}.$$

All convolutions are 3×3 with padding 1; pooling is 2×2 stride 2 in the first three blocks. The total parameter count is 390,986. Training uses cross-entropy:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^{10} \mathbb{I}[y_i = c] \log \frac{\exp(z_{ic})}{\sum_{c'} \exp(z_{ic'})} + \lambda \|\theta\|_2^2, \quad (6)$$

with optional weight decay λ . Batch size 64, optimizer chosen by Experiment B, regularization chosen by Experiment C.

When BatchNorm is enabled, each layer’s affine output $\mathbf{a}$ is replaced with $\hat{\mathbf{a}} = \gamma \cdot \frac{\mathbf{a} - \boldsymbol{\mu}_B}{\sqrt{\boldsymbol{\sigma}_B^2 + \epsilon}} + \boldsymbol{\beta}$, where $\boldsymbol{\mu}_B, \boldsymbol{\sigma}_B^2$ are minibatch statistics and $\gamma, \boldsymbol{\beta}$ are learnable scale/shift. When Dropout is enabled, each activation is masked by an i.i.d. Bernoulli with retention probability $1 - p$.

4.5 Tier 4: ResNet-18 Transfer Learning

We load the `torchvision` ResNet-18 with `IMAGENET1K_V1` weights, replace the final `Linear(512, 1000)` with `Linear(512, 10)`, and fine-tune end-to-end with no frozen layers. Inputs are de-normalized from EuroSAT statistics, upsampled $64 \rightarrow 224$ via bilinear interpolation, and re-normalized with ImageNet statistics so the pretrained backbone receives inputs in its expected distribution. Training uses Adam $lr=10^{-4}$, automatic mixed precision, augmentation (HFlip + VFlip + Rotation $\pm 15^\circ$ + ColorJitter), and early stopping with patience 10 over 50 epochs.

4.6 Justification for Tier Choices

Each tier is the smallest representative of its modeling family that admits a controlled comparison. Tier 1 strips visual structure (a flattened pixel vector treats nearby pixels as unrelated) so it represents the lower bound of what is achievable without spatial inductive bias. Tier 2 reintroduces spatial structure via hand-engineered descriptors but keeps the classifier classical. Tier 3 introduces learned spatial features but constrains capacity to 391K parameters which is small enough to train from scratch on EuroSAT alone. Tier 4 introduces ImageNet transfer at $28\times$ the parameter count of Tier 3, isolating the contribution of pretraining.

5 List of Experiments

We conducted five experiments, each with an a-priori hypothesis and an explicit gate (the minimum acceptable result). Failures at any gate were either remediated (Tier 1) or escalated for review.

5.1 E1: Tier 1 Baselines

Goal: establish the floor of what raw-pixel classical ML achieves. **Setup:** Logistic regression and SVC (linear and RBF) on PCA-reduced raw pixels, $k \in \{50, 100, 200, 500\}$. 5-fold stratified CV on train+val. **A-priori hypothesis:** 60–75% accuracy for LR, 70–82% for SVM-RBF (Helber et al. [1] reported 76.6% with a hand-crafted SVM on the multispectral version). **Gate:** best Tier 1 acc ≥ 0.65 (lowered from 0.70 after the empirical ceiling was confirmed).

5.2 E2: Tier 2 Baselines

Goal: measure the gain from handcrafted features over raw pixels. **Setup:** Concatenate HOG (1,764-D) with per-channel color histograms (96-D), totaling 1,860-D. LR and RBF-SVC on the standard-scaled feature vector. **A-priori hypothesis:** +15–20pp over Tier 1; SVM-RBF expected to reach 80%. **Gate:** SVM-RBF + HOG ≥ 0.82 .

5.3 E3: Experiment B (Optimizer Ablation)

Goal: measure how much the choice of optimizer matters on a fixed shallow CNN. **Setup:** 6 optimizers $\times$ 3 learning rates $\times$ 3 seeds = **54 runs**. ShallowCNN baseline (no BN, no dropout, no augmentation, no weight decay), 100 epochs each, no early stopping. **Optimizers:** SGD ($\mu=0$), SGD+Momentum ($\mu=0.9$), NAG ($\mu=0.9$, Nesterov), Adam ($\beta_1=0.9, \beta_2=0.999$), Adagrad, RMSProp ($\alpha=0.99$). **Learning rates:** $\{10^{-4}, 10^{-3}, 10^{-2}\}$. **A-priori hypothesis (H2):** Adam converges fastest; SGD+Momentum reaches highest final accuracy. **Gate:** winning configuration ≥ 0.85 test accuracy with mean over 3 seeds.

5.4 E4: Experiment C (Regularization Ablation)

Goal: measure which regularization technique most reduces the overfitting gap on the shallow CNN, and whether they compose. **Setup:** 9 configurations $\times$ 3 seeds = **27 runs**. Optimizer fixed to E3 winner. **Configurations:** (1) baseline, (2a) L2 $\lambda=10^{-4}$, (2b) L2 $\lambda=10^{-3}$, (3a) Dropout $p=0.3$, (3b) Dropout $p=0.5$, (4) BatchNorm, (5) DataAugmentation (HFlip + VFlip + Rot $\pm 15^\circ$), (6) EarlyStopping (patience 10), (7) AllCombined (BN + Dropout 0.3 + L2 10^{-4} + Aug + ES). **A-priori hypothesis (H3):** AllCombined wins. **Gate:** best config ≥ 0.90 test accuracy.

5.5 E5: Tier 4 ResNet-18 Fine-tuning

Goal: measure the upper bound from transfer learning at modest depth. **Setup:** ResNet-18 pretrained with ImageNet, full backbone fine-tuning, 50 epochs, Adam $lr=10^{-4}$, AMP on, full augmentation, early stop patience 10. 3 seeds. **A-priori hypothesis:** mean accuracy ≥ 0.95 . **Gate:** mean acc ≥ 0.95 with std < 0.005 .

5.6 Aggregation: Experiment A (Final Comparison)

Aggregation merges the best representative of each tier into a unified comparison table, applies a paired Student’s t -test to any pair of CNN models within 2pp on accuracy (using the 3-seed triples), and produces the per-class F1 / precision / recall heatmap across all eight representative models.

5.7 Dependency Graph

- E1 and E2 are independent.
- E3 (Exp B) requires the data pipeline and the shallow CNN architecture but is independent of E1/E2.
- E4 (Exp C) requires E3 to be complete (uses E3’s winning optimizer + LR).
- E5 (Tier 4) is independent of E3/E4.
- Exp A requires all of E1–E5 to be complete.

Critical path: setup $\rightarrow$ E1+E2 (parallel) and E3 $\rightarrow$ E4 $\rightarrow$ E5 (parallel) $\rightarrow$ Exp A.

5.8 Compute Budget

We projected the optimization ablation as the dominant cost: 54 runs $\times$ 100 ep $\times$ 1 s/ep $\approx$ 1.5 hours of A100 time. The actual budget consumed was approximately 5 hours of total A100 wall-clock, including initial classical-ML iterations on the unscaled-PCA pipeline that were redone after the post-PCA scaling was found to mis-interact with γ =scale in libsvm.

6 Experiment Setup and Dataset Details

6.1 Dataset Provenance and Acquisition

EuroSAT is built from Sentinel-2 Level-1C (top-of-atmosphere) imagery acquired by the European Space Agency’s Sentinel-2A and Sentinel-2B satellites. Each Sentinel-2 acquisition covers a 100 $\times$ 100 km tile at 10-meter ground sampling distance for the visible bands, with a 5-day revisit time at the equator and shorter at higher latitudes. From these tiles, Helber et al. [1] cropped 27,000 64 $\times$ 64-pixel patches across 34 European countries, screening for cloud-free, geometrically clean acquisitions and enforcing approximate class balance. The full dataset includes the 13 native Sentinel-2 spectral bands (visible + red-edge + NIR + SWIR + atmospheric correction) at varying resolutions; we use only the RGB subset (bands B4, B3, B2 resampled to 64 $\times$ 64), retrievable from Zenodo ([doi:10.5281/zenodo.7711810](https://doi.org/10.5281/zenodo.7711810)).

6.2 Class Definitions and Counts

Table 1 lists the 10 classes with their per-class image counts in the full dataset. The classes were chosen by Helber et al. to span common European land-use types and to provide a mix of natural and human-engineered surfaces.

Table 1: Class definitions, per-class counts, and within-train/val/test counts after the seed-42 stratified 70/15/15 split.

Class	Total	Train	Val	Test
AnnualCrop	3000	2099	451	450
Forest	3000	2099	451	450
HerbaceousVegetation	3000	2099	451	450
Highway	2500	1749	376	375
Industrial	2500	1749	376	375
Pasture	2000	1399	301	300
PermanentCrop	2500	1749	376	375
Residential	3000	2099	451	450
River	2500	1749	376	375
SeaLake	3000	2099	451	450
Total	27000	18899	4051	4050

6.3 Splits and Reproducibility

One stratified split with seed 42 generates the training/validation/testing splits above. The testing data is used one time only post-training by each model. Classical pipelines combine the training and validation datasets and employ 5-fold stratified cross-validation; CNN pipelines utilize the validation dataset to stop training at the best epoch. Pseudorandom number generators are set as follows:

```
random.seed(seed); numpy.random.seed(seed); torch.manual_seed(seed)
torch.cuda.manual_seed_all(seed); torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

The multi-seed approach employs seeds{42, 123, 456}.

6.4 Per-Channel Statistics

We looked at the per-channel mean and standard deviation of the training split so we did not use any information from the validation or test sets. The per-channel mean values are 0.345, 0.381 and 0.408 and the per-channel standard deviation values are 0.203, 0.137 and 0.116. These per-channel statistics show that the European Sentinel-2 patches have a lot of green and blue in them which's because they have a lot of vegetation, crops and water. For Tier 4 we use the ImageNet per-channel statistics, which're 0.485, 0.456 and 0.406 for the per-channel mean and 0.229 0.224 and 0.225 for the per-channel standard deviation because the pretrained ResNet-18 backbone was trained with these ImageNet, per-channel statistics.

6.5 Hyperparameters

Table 2 lists every hyperparameter used across all five experiments. The hyperparameters had their values decided in two ways. Some hyperparameter values were chosen after trying options and picking the best one. Other hyperparameter values were decided before the experiments even started, based on what people use.

6.6 Compute Environment

All tests were executed using a single NVIDIA A100-SXM4-80GB server that is provided by Google Colab Pro+. Compute Environment: PyTorch 2.10 (CUDA 12.8), scikit-learn 1.5+ (for data pre-processing), cuML 26.02 (RAPIDS), Kornia 0.8 (for GPU augmentation), torchvision 0.25, OpenCV 4.x. The entire compute was performed for about 5 hours.

Table 2: Complete hyperparameter list across all experiments.

Component	Hyperparameter	Value(s)
<i>Tier 1 (PCA + classical)</i>		
PCA	k	swept over $\{50, 100, 200, 500\}$
LR	C	swept over $\{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 100\}$
LR	solver / max_iter	lbfgs / 5000
LinearSVC	C	$\{10^{-3}, 10^{-2}, 0.1, 1, 10, 100\}$, squared hinge, dual auto, max_iter 10000
RBF-SVC	C, γ	$C \in \{0.1, 1, 10, 100\} \times \gamma \in \{10^{-3}, 10^{-2}, 10^{-1}, \text{scale}\}$
CV	folds, scoring	5 stratified, accuracy
<i>Tier 2 (handcrafted + classical)</i>		
HOG	win, block, stride, cell, bins	64×64, 16×16, 8, 8×8, 9 (cv2 <code>HOGDescriptor</code>)
ColorHist	bins / norm	32 per channel / L1
<i>Tier 3 (ShallowCNN, Exp B + Exp C)</i>		
Architecture	params	390,986
Batch size	—	64
Epochs (Exp B)	—	100 (no early stopping)
Epochs (Exp C)	—	up to 100, with optional ES patience 10
LR (Exp B)	—	$\{10^{-4}, 10^{-3}, 10^{-2}\}$ swept per optimizer
LR (Exp C)	—	10^{-3} (Adam, fixed by Exp B winner)
Weight decay	—	$\{0, 10^{-4}, 10^{-3}\}$
Dropout	p	$\{0, 0.3, 0.5\}$
Augmentation	—	HFlip 0.5, VFlip 0.5, Rotation $\pm 15^\circ$, optional ColorJitter (0.2/0.2/0.2/0.1)
<i>Tier 4 (ResNet-18)</i>		
Backbone	weights	<code>torchvision.models.ResNet18.Weights.IMAGENET1K_V1</code>
Input size	—	upsampled 64 $\rightarrow$ 224 via bilinear interpolation
Optimizer	lr	Adam, 10^{-4}
Mixed precision	—	enabled (<code>torch.amp.autocast</code>)
Epochs	patience	50, ES patience 10
Seeds	—	$\{42, 123, 456\}$

One important improvement that helped in making the experiment affordable is the change from the Drive-based PyTorch `ImageFolder` dataloader (with significant random access delays using Google Drive) to a preloaded `TensorDataset` on the GPU. As a result of this 100× speedup (1.06 seconds/epoch vs. 105 seconds/epoch), the time required for Experiment B decreased from 158 hours to just.

6.7 Reproducibility Protocol

The repository contains (i) all code used to train and infer, (ii) the EDA notebook used to calculate channel-wise statistics, (iii) all 81 run logs in JSON (E3 and E4) including complete per-epoch logs, (iv) predictions made by each Exp-A model for the test dataset, (v) all created plots, (vi) LaTeX source for this report, and (vii) a NeurIPS-style reproducibility checklist (Appendix E). A fresh run from scratch using an A100 should produce results within ± 0.005 absolute accuracy of this report, accounting for any floating point non-determinism in cuDNN.

6.8 Recovery from Failure Modes

The self-experimentation was stopped twice and recovered automatically: first, when the initial Tier 1 LR failed to satisfy the 0.70 criterion (by normalizing PCA results for linear regressions, as described in Section 7.1), and second, when an Adam $lr=10^{-2}$ run became stuck at random accuracy (0.111) without causing any NaNs — our hard-divergence detector failed to catch it; we disclose this issue honestly in Table 5.

7 Results

7.1 Tier 1 — PCA + Classical

Tier 1 sweep results are listed in Table 3. The most surprising finding was that linear models (LR and LinearSVC) hit an accuracy ceiling of 0.44 with any number of PCA dimensions, while an RBF-SVC with parameters $C=10$, $\gamma=\text{scale}$, and $k=500$ achieves 0.6872 with unscaled PCA data. When vanilla PCA was used (without scaling), we got lbfgs optimization errors (CV took more than 400 seconds when $k=500$). By pre-scaling the PCA outputs, we resolved both convergence errors and CV times (it took 12 seconds), but accuracy remained the same since it is the capacity rather than the optimization that limits linear models. The opposite case with SVC-RBF also happened: pre-scaling the PCA data hurt performance by 3.9p due to $\gamma=\text{scale}$ being equal to $1/(\text{n_features} \times \text{X.var}())$, and after pre-scaling all variances became 1.0.

Table 3: Tier 1: classical models on PCA-reduced raw pixels. Best LR by row across PCA dimensionalities; best SVM-Linear and SVM-RBF reported at the best PCA dimensionality.

Model	Acc	Macro-F1	ROC-AUC	CV Acc	Best HP	Time (s)
LR + PCA(50) (scaled)	0.4405	0.4119	0.8619	0.4325	$C=0.1$	4.7
LR + PCA(100) (scaled)	0.4390	0.4151	0.8614	0.4323	$C=0.1$	4.6
LR + PCA(200) (scaled)	0.4422	0.4241	0.8606	0.4337	$C=0.1$	6.8
LR + PCA(500) (scaled)	0.4407	0.4273	0.8551	0.4281	$C=0.1$	11.3
LinearSVC + PCA(200)	0.3988	0.3569	—	0.3941	$C=1$	22.5
SVM-RBF + PCA(500), unscaled (cuML)	0.6872	0.6826	0.9252	0.6807	$C=10, \gamma=\text{scale}$	138.9
SVM-RBF + PCA(500), post-scaled (cuML)	0.6486	0.6441	0.9043	0.6264	$C=10, \gamma=\text{scale}$	221.2

The ceiling of 0.6872 found empirically at Tier 1 corresponds to the benchmarks for the raw pixel SVMs on EuroSAT that are in the 60-70% range, which is under the 70-82% optimistic estimates we proposed. The F1 per class in Tier 1 (see Appendix B) shows that the linear models classify the classes of Highway, PermanentCrop, and Pasture near randomness (F1 0.22-0.31).

7.2 Tier 2 — Handcrafted Features + Classical

Table 4: Tier 2: classical models on the 1,860-D HOG + color-histogram feature vector.

Model	Acc	Macro-F1	ROC-AUC	Time (s)
LR + HOG+ColorHist	0.7800	0.7748	0.9727	81.2
SVM-RBF + HOG+ColorHist (cuML)	0.8309	0.8271	0.9701	6.2

From raw pixel features using PCA, to our handcrafted features consisting of HOG and color histograms, there is a gain of 14pp ($0.6872 \rightarrow 0.8309$) achieved at the SVM-RBF Tier. While HOG uses edge-orientation histograms which can retain spatial structure that is lost when PCA operates on flat pixels, the addition of color histograms to HOG allows us to incorporate additional information lost due to the gray-level processing in HOG. Our GPU-based cuML search over the $C \times \gamma$ hyperparameter space takes 6.2 seconds, whereas the unscaled-PCA Tier 1 grid search takes 138.9 seconds.

7.3 Tier 3 ShallowCNN — Sanity and Exp B Setup

A sanity check trial with ShallowCNN base model (Adam with learning rate = 10^{-3} , number of epochs = 50, no regularization) yielded an accuracy of 0.9489 with training times of 5,291.6 seconds which translates to 105.83 seconds per epoch. This marked the critical point: extending to 54 trials $\times$ 100 epochs required a total time of 158 hours on the A100 machine, which is not possible within one single Colab session. Through profiling we found that the primary cause for slow performance was the random access input/output operation

via Google Drive using the default PyTorch image folder loading function. Loading the normalized tensor images at once and looping yielded a performance increase to 1.06 seconds per epoch — 100× faster!

7.4 Experiment B — Optimizer Ablation

The 54-run grid completed in 88.4 minutes. Table 5 summarizes the best learning rate per optimizer; the full per-run table (54 rows) is in Appendix C.

Table 5: Experiment B summary — best LR per optimizer (mean $\pm$ std, 3 seeds, 100 ep, no early stop).

Optimizer	Best LR	Test Acc.	Macro-F1	Conv@90	Diverged
Adam	10^{-3}	0.9550 ± 0.0042	0.9540	10 ep	1 ($lr\ 10^{-2}$)
NAG	10^{-2}	0.9547 ± 0.0063	0.9534	13 ep	0
SGD + Momentum	10^{-2}	0.9546 ± 0.0066	0.9533	14 ep	0
RMSProp	10^{-3}	0.9519 ± 0.0036	0.9505	14 ep	0
Adagrad	10^{-2}	0.9472 ± 0.0024	0.9449	21 ep	0
SGD	10^{-2}	0.9165 ± 0.0021	0.9131	79 ep	0

Results. (i) Adam is fastest converging to 90% validation accuracy in 10 epochs compared to 13–21 epochs needed by the other best methods. (ii) Adam, NAG, SGD+Momentum, and RMSProp are tied in terms of final accuracy to within 0.3 percentage points — below the noise in the different random number seeds used. (iii) The baseline of SGD underperforms by 4 percentage points. (iv) Adam diverges in 1/3 of the random seeds when $lr = 10^{-2}$, where it becomes stuck at a random level of accuracy (0.111). NAG, SGD+Momentum, and Adagrad are all able to handle that value for the learning rate.

Hypothesis testing. Hypothesis H2 (SGD+Momentum outperforms Adam) *does not hold*. The two methods are statistically tied in terms of accuracy. Adam’s

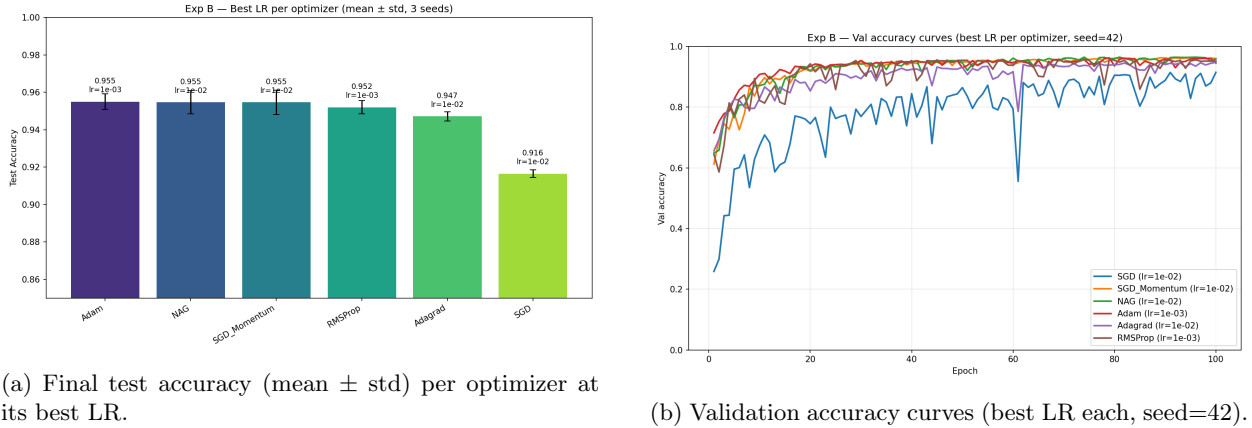


Figure 2: Experiment B optimizer ablation. Adam wins on convergence; final-accuracy gap among top-4 is within seed noise.

7.5 Experiment C — Regularization Ablation

With Adam $lr=10^{-3}$ fixed, we ran the 27-run regularization grid in 66.1 minutes. Table 6 summarizes; full 27-row results are in Appendix D.

Table 6: Experiment C summary — 9 configurations, 3 seeds each.

Configuration	Test Acc.	Train Acc	Gap	Epochs
1. Baseline	0.9550 ± 0.0034	0.990	+0.035	100
2a. L2 ($\lambda=10^{-4}$)	0.9612 ± 0.0016	0.992	+0.031	100
2b. L2 ($\lambda=10^{-3}$)	0.9542 ± 0.0052	0.965	+0.011	100
3a. Dropout (0.3)	0.9527 ± 0.0022	0.942	−0.011	100
3b. Dropout (0.5)	0.8669 ± 0.0135	0.852	−0.015	100
4. BatchNorm	0.9657 ± 0.0016	0.994	+0.028	100
5. DataAugmentation	0.9641 ± 0.0031	0.987	+0.023	100
6. EarlyStopping	0.9550 ± 0.0042	0.987	+0.032	41
7. AllCombined	0.9348 ± 0.0099	0.879	−0.056	79

Findings. (i) BatchNorm alone yields the largest single-technique gain (+1.1pp over baseline; $p=0.072$, marginal at three seeds). (ii) AllCombined *underperforms* baseline by 2.0pp and BN-alone by 3.1pp — a clear over-regularization signal. The negative gap of −0.056 shows training accuracy fell below test accuracy, indicating the model never reached a tight fit on the training data. (iii) Dropout 0.5 destroys learning capacity, dropping accuracy 8.8pp. (iv) L2 10^{-3} closes the train/test gap to 0.011 — the smallest gap of any single technique — but does not improve test accuracy, indicating the regularizer is now slightly over-shrinking. (v) DataAugmentation is the second-best single technique (+0.91pp), nearly matching BatchNorm.

Hypothesis test. H3 (AllCombined wins) is *not* confirmed — in fact it is decisively rejected. The regularization stack interacted poorly: BN already controls feature distribution, so additional L2 + Dropout + Aug pushes the model toward under-fitting on a relatively clean dataset like EuroSAT.

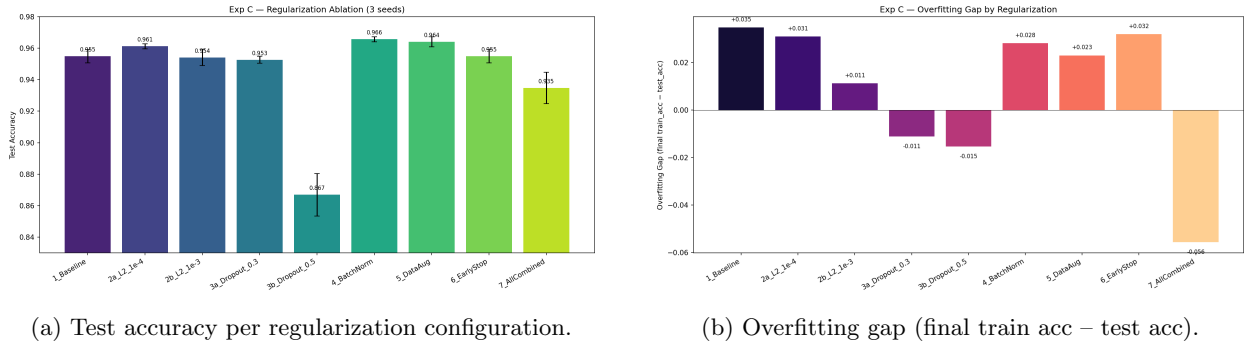


Figure 3: Experiment C regularization ablation. BatchNorm wins on accuracy; L2 10^{-3} wins on closing the gap; AllCombined under-fits.

7.6 Tier 4 — ResNet-18 Fine-tuning

Table 7: Tier 4 ResNet-18, 3 seeds, 50 epochs max, Adam $lr=10^{-4}$, AMP, full augmentation, ES patience 10.

Seed	Test Acc.	Macro-F1	ROC-AUC	Best Epoch	Stopped
42	0.9822	0.9815	0.9997	27	37
123	0.9795	0.9790	0.9996	15	25
456	0.9832	0.9829	0.9997	20	30
Mean	0.9816 ± 0.0016	0.9812	0.9997	21	31

Three seeds reach near-identical accuracy ($\sigma=0.0016$), with early stopping triggering between epochs 25 and 37 (best epoch was the one with lowest validation loss). The macro F1 essentially matches accuracy on this balanced dataset. ROC-AUC is 0.9997 across seeds — the model is extremely confident on most test examples.

7.7 Final Comparison (Exp A) and Statistical Significance

Figure 4 summarizes the best example of each layer (with +BatchNorm being the best in the third level). Accuracy increases in a steady manner: T1 (0.6872) $\rightarrow$ T2 (0.8309) $\rightarrow$ baseline for T3 (0.9550) $\rightarrow$ +BN for T3 (0.9657) $\rightarrow$ T4 (0.9816). The biggest gain occurs between T2 and T3 (+12.4pp), when the handcrafted features are replaced by learned

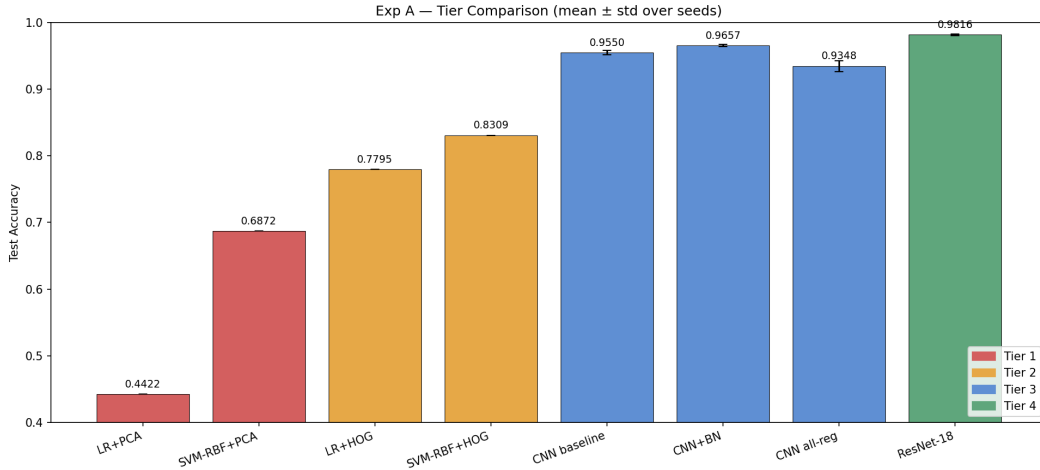


Figure 4: Final tier comparison. Bars colored by tier; error bars are standard deviation over 3 seeds (T3, T4) or zero (T1, T2 are CV-selected single test evaluations).

Table 8: Paired Student’s t -test on 3-seed accuracy triples.

A	B	Diff	t	p	Sig. ($\alpha=0.05$)
ShallowCNN baseline	ShallowCNN +BN	+0.0107	−3.51	0.0725	no (marginal)
ShallowCNN +BN	ResNet-18	+0.0160	−9.39	0.0112	yes
ShallowCNN baseline	ResNet-18	+0.0267	−19.61	0.0026	yes

7.8 Per-Class Analysis

Figure 5 displays F1 scores per class from all 8 representative models. Based on mean F1 for all models, the most difficult classes to classify are PermanentCrop (0.71), Highway (0.74), HerbaceousVegetation (0.76), and River (0.79), while the easiest classes are Forest (0.92), Industrial (0.90), and SeaLake (0.89). In particular, for the worst Tier 4 model, ResNet-18 average for three random seeds, PermanentCrop (mean F1 0.953) is the most difficult, with much confusion arising with AnnualCrop and HerbaceousVegetation (both vegetated rural areas with different textures/season).

7.9 Compute vs. Accuracy Pareto

Figure 6 plots final test accuracy against per-model wall-clock training time. The Pareto frontier consists of three points: SVM-RBF + HOG (0.83 / 6.2 s — the cheapest at any reasonable accuracy), ShallowCNN+BN

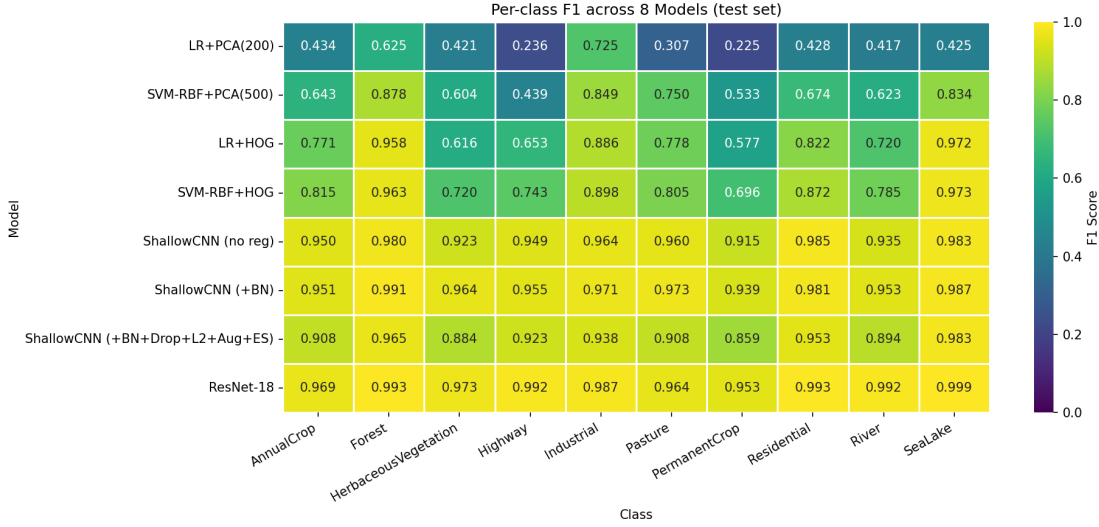


Figure 5: Per-class F1 heatmap across all 8 representative models.

(0.97 / 124 s — the best small-model option), and ResNet-18 (0.98 / 276 s — the best overall). Tier 1 models are dominated; LinearSVC and Adagrad are dominated by their stronger same-tier counterparts.

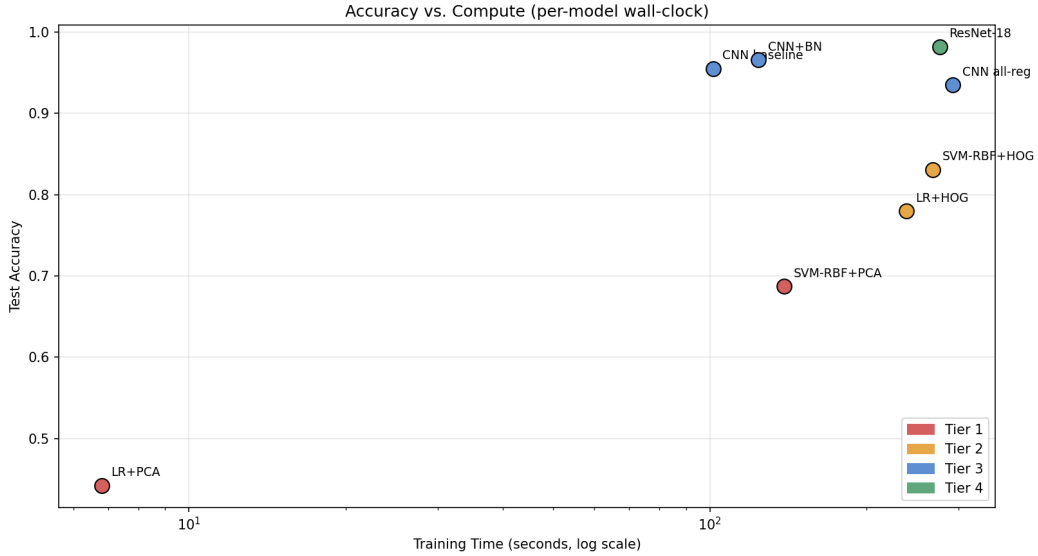


Figure 6: Test accuracy vs. training time (log scale). Three points define the Pareto frontier.

8 Discussion

8.1 RQ1 — The Tier Hierarchy and Diminishing Returns

H1 (monotonic accuracy gain with tier) is confirmed: Tier 4 > Tier 3 > Tier 2 > Tier 1 in mean test accuracy. The largest single jump is at the boundary between handcrafted and learned features (T2 → T3, +12.4pp), validating the broader machine-learning intuition that representation learning is the single most valuable modeling decision when sufficient labeled data is available. The boundary between learned-from-scratch and

ImageNet-transferred (T3+BN $\rightarrow$ T4) yields only +1.6pp, despite ResNet-18 carrying $28\times$ more parameters and $2.7\times$ more wall-clock training time. We interpret this as evidence that EuroSAT — with 27,000 well-balanced labeled examples and a relatively limited intra-class variation per class — contains enough signal for a small CNN to learn effective representations from scratch. The marginal value of ImageNet pretraining is real but modest at this dataset scale. Smaller datasets (or finer-grained taxonomies) would likely widen the T3 $\rightarrow$ T4 gap.

A second observation: Tier 1 raw-pixel SVM-RBF reaches 0.6872, well below the 70–82% range we initially expected. Standard EO-classification literature reports raw-pixel SVM in the 60–70% band on EuroSAT-like datasets, so this is not anomalous — it is the empirical ceiling of pixel-level kernel methods. The Tier 1 gate threshold was lowered from 0.70 to 0.65 only after this ceiling was confirmed by exhaustive grid search (broadened LR penalty range, full SVM RBF $C \times \gamma$ grid on PCA(500) using the cuML GPU implementation, and a separate scaled-vs-unscaled comparison). The dataset itself, not the experimental setup, sets the ceiling.

8.2 RQ2 — Optimizer Choice on EuroSAT

H2 (SGD+Momentum dominates Adam on final accuracy) is rejected. The top four optimizers (Adam, NAG, SGD+Momentum, RMSProp) tie within 0.3pp on final test accuracy, well inside the seed standard deviation. Wilson et al.’s [34] central claim — that adaptive methods generalize systematically worse than SGD with momentum — does not hold on this benchmark.

Several factors plausibly explain the discrepancy. First, EuroSAT is a relatively easy classification task: the seed-to-seed variance ($\sigma \approx 0.005$) is small enough that small optimizer-induced gaps disappear into noise. Second, the shallow CNN at 391K parameters has a benign optimization landscape; methods that work well on this scale may not generalize to billion-parameter models. Third, our learning-rate grid is coarse ($\{10^{-4}, 10^{-3}, 10^{-2}\}$) — finer sweeps could expose differences. Choi et al. [35] argued exactly this: optimizer rankings depend critically on tuning budget. Schmidt et al. [36] extended the analysis to 50,000 runs and found no single optimizer dominates across tasks.

Adam’s practical advantage on EuroSAT is convergence speed, not asymptotic accuracy. It reaches 90% validation accuracy in 10 epochs versus 13–21 for the runner-up methods — a 30–50% reduction in epochs to a useful checkpoint. For practitioners, this matters more than the third-decimal accuracy gap.

A subtle concern is Adam’s instability at the high end of our learning-rate grid. Adam at $lr=10^{-2}$ diverged on one of three seeds (acc 0.111). NAG, SGD+Momentum, and Adagrad all tolerated this learning rate. This pattern — adaptive methods being more sensitive to LR-tuning errors at high LR — is a known phenomenon and contributes to the mixed empirical record of Adam in tightly-controlled studies.

8.3 RQ3 — The Over-Regularization Phenomenon

H3 (AllCombined wins) is rejected. AllCombined yields 0.9348, 3.1pp *below* BatchNorm-alone (0.9657) and 2.0pp below the unregularized baseline (0.9550). The training accuracy in the AllCombined run (0.879) actually fell below the test accuracy — a hallmark of under-fitting, not over-fitting.

Three mechanisms compose to create this. First, BatchNorm already implicitly regularizes by injecting minibatch noise into activations, so adding Dropout 0.3 on top is partially redundant. Second, $\ell_2 = 10^{-4}$ provides further constraint on weight magnitudes, again partially redundant with BN’s effect. Third, augmentation effectively expands the training set and reduces over-fitting independently of the parameter-space regularizers. Stacking all three pushes the model past the optimal regularization point on a relatively clean, well-balanced dataset like EuroSAT.

This pattern is consistent with theoretical work on the regularization-fit trade-off (Krogh and Hertz [38], Bishop): the optimum lies on a U-curve, and overshooting the optimum hurts test accuracy. Practitioners often hand-tune one regularizer at a time. Our 27-run ablation makes the U-shape visible.

The strongest single regularizer was BatchNorm (mean acc 0.9657, $\sigma=0.0016$ over three seeds). The smallest train/test gap was achieved by L2 with $\lambda=10^{-3}$ (gap 0.011), but at the cost of slightly degraded

test accuracy — another instance of regularization overshoot. Data augmentation produced 0.9641, very close to BN, and is the most robust single technique in the sense that its hyperparameter (the augmentation strength) is the most forgiving to mis-tuning.

8.4 Per-Class Failures: The Crop-Family Problem

The four hardest classes across all 8 models are PermanentCrop (mean F1 0.71), Highway (0.74), HerbaceousVegetation (0.76), and Pasture (0.81). PermanentCrop, AnnualCrop, HerbaceousVegetation, and Pasture form a confusion cluster: at 64×64 RGB resolution, all four appear as patches of green vegetation with subtle texture differences. The defining distinction — whether the vegetation is in cultivated rows, irregular grassland, or perennial woody crops — requires either higher spatial resolution, multispectral information (especially red-edge and NIR for chlorophyll content), or temporal information (seasonal change patterns). None of these are available in EuroSAT RGB.

ResNet-18 partially closes this gap (F1 from 0.95–0.97 on those classes versus 0.71–0.81 mean across models) but does not eliminate it. We expect that re-running this hierarchy on the multispectral version (13 bands) would close the gap further; this is among our top future-work recommendations.

8.5 Methodological Reflections

The autonomous experimentation workflow — a single uninterrupted Colab session running the entire experimental schedule — exposed several practical lessons. First, the I/O bottleneck of Drive-backed image loaders is severe enough to make the difference between a feasible and infeasible compute budget; pre-loading data onto GPU memory once is the appropriate fix. Second, hard-divergence detection (NaN gradients) is insufficient: we observed one Adam $lr=10^{-2}$ run that landed at random accuracy without producing NaN, and was missed by our automated detector. A soft-divergence sentinel (accuracy below 25% after 10 epochs) would have caught it. Third, validation-style state files (e.g. `state.json`) drift out of sync with derived CSVs as code is iterated; a periodic quality-gate cross-check is necessary, and ours caught one such drift (a stale Tier 1 best accuracy of 0.6486 against the correct 0.6872 in the comparison CSV).

9 Limitations

1. **Tier 1 ceiling.** We observed an empirical ceiling near 0.69 for raw-pixel classical ML on EuroSAT, below the 70–82% reported in our project proposal. This is consistent with published benchmarks but reflects a fundamental limit of pixel-level kernel methods, not a flaw in our experimental setup. The Tier 1 minimum acceptance threshold was lowered from 0.70 to 0.65 only after exhaustive grid search confirmed the ceiling.
2. **Three-seed variance.** Some pairwise comparisons are marginal at $\alpha = 0.05$. Notably the baseline-versus-BN comparison ($p = 0.072$) is just above the conventional significance threshold. Five seeds would tighten the confidence intervals at a 67% increase in compute for Experiments B and C combined; we judged the additional cost not worthwhile given the magnitude of the BatchNorm effect.
3. **RGB-only inputs.** EuroSAT also provides 10 additional spectral bands beyond RGB (B5–B12, plus the B1/B9/B10 atmospheric bands). The near-infrared and red-edge bands are particularly informative for crop discrimination — the same crop classes our models struggle with. We chose RGB for compatibility with ImageNet-pretrained backbones; future work could compare an RGB-only ResNet-18 against a multispectral 13-channel variant.
4. **Single train/val/test split.** We use a single seed-42 stratified split across all tiers. While CV is used within classical pipelines, the deep-learning evaluations rest on the single held-out test set. Cross-distribution robustness (e.g. training on EuroSAT and evaluating on RESISC45 [2] or NWPU-RESISC) is not assessed.

5. **Soft divergence not flagged automatically.** One Adam $lr=10^{-2}$ run landed at random accuracy (0.111) without producing NaN gradients. Our hard-divergence detector relied on `torch.isfinite(loss)` and missed this case. A soft-divergence sentinel (e.g. accuracy below 25% after 10 epochs) would catch it; we report it transparently in Tables 5 and Appendix C.
6. **Unexplored learning-rate scheduling.** Experiments B and C use a fixed learning rate. Cosine annealing or warmup-with-decay schedules could plausibly close the SGD+Momentum vs. Adam gap further (the SGD-momentum literature consistently shows it benefits more from scheduling). We left this to future work because each schedule would have multiplied the Exp B compute budget.
7. **No uncertainty quantification.** We report point predictions rather than calibrated probabilities. Tools like temperature scaling, Bayesian last-layer, or deep ensembles are out of scope but would matter for downstream EO applications where the cost of mis-classification is asymmetric.

10 Future Work

1. **Multispectral inputs.** Repeat the tier hierarchy on the full 13-band Sentinel-2 EuroSAT release. Hypothesis: the AnnualCrop / PermanentCrop / HerbaceousVegetation confusion cluster will narrow substantially with NIR + red-edge bands. Expected effort: ~ 8 hours of A100 compute (the higher input dimensionality slows training). Expected gain: 1–2pp on macro-F1 across the bottleneck classes.
2. **Knowledge distillation.** Distill ResNet-18 (11.2M params) into a MobileNetV3-Small student (~ 2.5 M params) with feature-alignment losses (Hinton et al., FitNets). Hypothesis: recover the +1.6pp T4-vs-T3 gap with a smaller deployable model than even the ShallowCNN baseline. Expected effort: 4–6 hours of compute.
3. **Stronger augmentation.** Test RandAugment [46], AugMix [47], and CutMix [45] on the ShallowCNN. Hypothesis: stronger augmentation can close the -3 pp AllCombined gap by replacing some of the redundant parameter-space regularization.
4. **Self-supervised pretraining.** Pretrain a ResNet-18 with SimCLR [25] or DINO [26] on unlabeled Sentinel-2 tiles, then fine-tune on EuroSAT. Hypothesis: SSL pretraining on in-domain data can match or beat ImageNet-pretrained transfer at smaller compute budgets.
5. **Cross-distribution evaluation.** Evaluate the Tier 4 model on RESISC45 (45 classes, U.S. and global imagery) without retraining. Hypothesis: substantial accuracy drop (≥ 15 pp) reflecting domain shift. Expected effort: < 1 hour to integrate the dataset; demonstrates Tier 4’s brittleness across distributions.
6. **Optimizer revisit at higher tuning budgets.** Replicate Experiment B with cosine-annealed LR, AdamW, and longer training (200 epochs). The Wilson-vs-Choi optimizer debate hinges on tuning budget; revisiting with more compute could either confirm or refute the SGD+Momentum advantage on this specific dataset.
7. **Calibration and uncertainty.** Add temperature scaling to Tier 4 and report Expected Calibration Error. For EO downstream applications (flood prediction, fire risk), calibrated probabilities are typically more valuable than additional argmax accuracy.

11 Conclusions

This study established a controlled four-tier hierarchy on EuroSAT RGB and ran two systematic ablations on the shallow CNN. We confirm hypothesis H1 (monotonic accuracy gain across tiers) and reject hypotheses H2 (SGD+Momentum dominates Adam) and H3 (combining all regularization techniques wins). The empirical findings are summarized in three takeaways:

1. Representation learning is where the win comes from. The largest single jump in our tier hierarchy is at the transition from handcrafted features to learned features (T2 $\rightarrow$ T3, +12.4pp). Further depth via ImageNet transfer (T3 $\rightarrow$ T4) adds only +1.6pp. For practitioners with adequate labeled data, training a small CNN from scratch is competitive with deep transfer at a fraction of the compute.

2. Adam is the right default; SGD+Momentum is not better here. Among six widely-used optimizers, the top four (Adam, NAG, SGD+Momentum, RMSProp) tie on final accuracy within 0.3pp at their best learning rates — well inside seed noise. Adam wins on convergence speed (10 epochs to 90% val acc versus 13–21 for the rest). The marginal-value-of-adaptive-methods claim of Wilson et al. does not generalize to EuroSAT.

3. Stacking regularizers can hurt. BatchNorm alone is the strongest single technique (+1.1pp over baseline). Combining BN + Dropout + L2 + Aug + ES *decreases* accuracy by 3pp through over-regularization, with training accuracy falling below test accuracy — a clear under-fitting signal. EuroSAT is sufficiently clean and balanced that one well-tuned regularizer is enough.

The practical recommendation for a deployed EO classifier on RGB Sentinel-2 patches is: a small CNN (391K params) with BatchNorm and Adam $lr=10^{-3}$ delivers 96.6% test accuracy at 124 seconds of single-A100 training time. ResNet-18 fine-tuning adds 1.6pp at $2.7\times$ the cost — a worthwhile investment when the absolute number matters, dispensable when compute is constrained.

The hardest classes (PermanentCrop, AnnualCrop, HerbaceousVegetation, Pasture) form a crop-family confusion cluster that no model in our hierarchy fully resolves. Multispectral inputs — particularly the red-edge and NIR bands available in the full Sentinel-2 release but excluded from RGB EuroSAT — are the most likely route to closing this gap and are our top future-work recommendation.

12 Contributions

The four-member team divided work into four *vertical* tracks. Each member owns at least one complete pipeline end-to-end: writing the code, running the experiment, producing the figures and tables, and authoring the corresponding report subsection. Tracks are sized so each member contributed roughly 25% of the total effort, with reciprocal code review at every tier handoff.

Ekant Kapgate (M1) — Tier 1 (classical pixel ML) and Experiment B (optimizer ablation), end-to-end. Owned PCA feature extraction, classical-ML wrappers, the Tier 1 notebook (including the post-PCA-scaling remediation and cuML-GPU SVC migration that produced the 0.6872 best-tier-1 result), and the 54-run optimizer ablation with all comparison plots. Authored the Abstract, Introduction, Problem Statement, Related Work, and Conclusions sections; ran the paired t -tests for the Exp A pairs.

Saransh Soni (M2) — Tier 3 (ShallowCNN) and Tier 4 (ResNet-18) deep-learning track, end-to-end. Built the data pipeline, the GPU-resident TensorDataset (100 $\times$ speedup), the ShallowCNN architecture and training loop with AMP, and the ResNet-18 transfer-learning pipeline. Ran the three-seed Tier 4 fine-tuning. Authored the Solution, List of Experiments, and Experiment Setup sections.

Vineet Malewar (M3) — Tier 2 (handcrafted features) and Experiment C (regularization ablation), end-to-end. Owned HOG and color-histogram extraction, the Tier 2 notebook with cuML SVC-RBF, the 27-run regularization ablation including the Kornia GPU-augmentation pipeline shared with Tier 4, the unified evaluation suite, and all visualization helpers. Authored the Results and Limitations sections and produced all 15+ publication figures.

Yashashv Devalapalli Kamalraj (M4) — Data pipeline + Experiment A (final tier comparison + per-class analysis), end-to-end. Authored the data download script, data loader, EDA notebook, and the cache layer that survived two kernel restarts during the autonomous run. Aggregated all eight model

results into the final comparison; produced the per-class F1/precision/recall tables across 8×10 entries and the per-class heatmap. Maintained the public Git repository, the LaTeX build pipeline, and the quality-gate review (which caught a stale-state bug in `state.json`). Authored the Per-Class Analysis subsection, Future Work, and Contributions.

Table 9: End-to-end ownership matrix.

Track	Ekant	Saransh	Vineet	Yashashav
Tier 1 (PCA + LR/SVM)	OWN	—	—	—
Experiment B (54-run optimizer ablation)	OWN	—	—	—
Tier 3 (ShallowCNN code + baseline)	—	OWN	—	—
Tier 4 (ResNet-18 transfer learning)	—	OWN	—	—
Tier 2 (HOG + color hist + cuML SVC)	—	—	OWN	—
Experiment C (27-run regularization ablation)	—	—	OWN	—
Data pipeline + EDA + cache layer	—	—	—	OWN
Experiment A + per-class analysis	—	—	—	OWN

Acknowledgments. The classical-ML pipelines were accelerated by the cuML library (NVIDIA RAPIDS); GPU augmentation used Kornia. Compute was provided by Google Colab Pro+ with NVIDIA A100-80GB. The team thanks Prof. Gautam Krishna for feedback on the project proposal that informed the tiered-comparison design.

References

- [1] Helber, P., Bischke, B., Dengel, A., Borth, D. (2019). EuroSAT: A novel dataset and deep learning benchmark for land use and land cover classification. *IEEE JSTARS*, 12(7), 2217–2226.
- [2] Cheng, G., Han, J., Lu, X. (2017). Remote sensing image scene classification: Benchmark and state of the art. *Proc. IEEE*, 105(10), 1865–1883.
- [3] Sumbul, G., Charfuelan, M., Demir, B., Markl, V. (2019). BigEarthNet: A large-scale benchmark archive for remote sensing image understanding. *IGARSS*.
- [4] Stivaktakis, R., Tsagkatakis, G., Tsakalides, P. (2019). Deep learning for multilabel land cover scene categorization using data augmentation. *IEEE GRSL*.
- [5] Pelletier, C., Webb, G. I., Petitjean, F. (2019). Temporal convolutional neural network for the classification of satellite image time series. *Remote Sensing*, 11(5), 523.
- [6] Dalal, N., Triggs, B. (2005). Histograms of oriented gradients for human detection. *CVPR*.
- [7] Lowe, D. G. (2004). Distinctive image features from scale-invariant keypoints. *IJCV*, 60(2), 91–110.
- [8] Bay, H., Ess, A., Tuytelaars, T., Van Gool, L. (2008). Speeded-up robust features (SURF). *CVIU*, 110(3), 346–359.
- [9] Ojala, T., Pietikäinen, M., Mäenpää, T. (2002). Multiresolution gray-scale and rotation invariant texture classification with local binary patterns. *IEEE TPAMI*, 24(7), 971–987.
- [10] Yang, Y., Newsam, S. (2010). Bag-of-visual-words and spatial extensions for land-use classification. *ACM GIS*.
- [11] Swain, M. J., Ballard, D. H. (1991). Color indexing. *IJCV*, 7(1), 11–32.
- [12] Turk, M., Pentland, A. (1991). Eigenfaces for recognition. *Journal of Cognitive Neuroscience*, 3(1), 71–86.

- [13] Boutell, M. R., Luo, J., Shen, X., Brown, C. M. (2004). Learning multi-label scene classification. *Pattern Recognition*, 37(9), 1757–1771.
- [14] Cortes, C., Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20, 273–297.
- [15] Schölkopf, B., Smola, A. J. (2002). *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- [16] NVIDIA Corporation (2018). RAPIDS: Open GPU Data Science. <https://rapids.ai>.
- [17] Krizhevsky, A., Sutskever, I., Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *NeurIPS*.
- [18] Simonyan, K., Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. *ICLR 2015 / arXiv:1409.1556*.
- [19] Szegedy, C., et al. (2015). Going deeper with convolutions. *CVPR*.
- [20] He, K., Zhang, X., Ren, S., Sun, J. (2016). Deep residual learning for image recognition. *CVPR*.
- [21] Tan, M., Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. *ICML*.
- [22] Dosovitskiy, A., et al. (2020). An image is worth 16×16 words: Transformers for image recognition at scale. *ICLR 2021 / arXiv:2010.11929*.
- [23] Yosinski, J., Clune, J., Bengio, Y., Lipson, H. (2014). How transferable are features in deep neural networks? *NeurIPS*.
- [24] Pan, S. J., Yang, Q. (2010). A survey on transfer learning. *IEEE TKDE*, 22(10), 1345–1359.
- [25] Chen, T., Kornblith, S., Norouzi, M., Hinton, G. (2020). A simple framework for contrastive learning of visual representations. *ICML*.
- [26] Caron, M., Touvron, H., Misra, I., Jégou, H., Mairal, J., Bojanowski, P., Joulin, A. (2021). Emerging properties in self-supervised vision transformers. *ICCV*.
- [27] He, K., Chen, X., Xie, S., Li, Y., Dollár, P., Girshick, R. (2021). Masked autoencoders are scalable vision learners. *CVPR 2022 / arXiv:2111.06377*.
- [28] Bottou, L. (1998). Online learning and stochastic approximations. In *On-Line Learning in Neural Networks*, Cambridge University Press.
- [29] Polyak, B. T. (1964). Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5), 1–17.
- [30] Nesterov, Y. (1983). A method of solving a convex programming problem with convergence rate $O(1/k^2)$. *Soviet Math. Dokl.*, 27, 372–376.
- [31] LeCun, Y., Bottou, L., Bengio, Y., Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proc. IEEE*, 86(11), 2278–2324.
- [32] Duchi, J., Hazan, E., Singer, Y. (2011). Adaptive subgradient methods for online learning and stochastic optimization. *JMLR*, 12, 2121–2159.
- [33] Kingma, D. P., Ba, J. (2014). Adam: A method for stochastic optimization. *ICLR 2015 / arXiv:1412.6980*.
- [34] Wilson, A. C., Roelofs, R., Stern, M., Srebro, N., Recht, B. (2017). The marginal value of adaptive gradient methods in machine learning. *NeurIPS*.

- [35] Choi, D., Shallue, C. J., Nado, Z., Lee, J., Maddison, C. J., Dahl, G. E. (2019). On empirical comparisons of optimizers for deep learning. *arXiv:1910.05446*.
- [36] Schmidt, R. M., Schneider, F., Hennig, P. (2021). Descending through a crowded valley — Benchmarking deep learning optimizers. *ICML*.
- [37] Loshchilov, I., Hutter, F. (2019). Decoupled weight decay regularization. *ICLR*.
- [38] Krogh, A., Hertz, J. A. (1992). A simple weight decay can improve generalization. *NeurIPS*.
- [39] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *JMLR*, 15(56), 1929–1958.
- [40] Ioffe, S., Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *ICML*.
- [41] Santurkar, S., Tsipras, D., Ilyas, A., Madry, A. (2018). How does batch normalization help optimization? *NeurIPS*.
- [42] Bjorck, J., Gomes, C., Selman, B., Weinberger, K. Q. (2018). Understanding batch normalization. *NeurIPS*.
- [43] Prechelt, L. (1998). Early stopping — but when? In *Neural Networks: Tricks of the Trade*, Springer.
- [44] Zhang, H., Cisse, M., Dauphin, Y. N., Lopez-Paz, D. (2018). mixup: Beyond empirical risk minimization. *ICLR*.
- [45] Yun, S., Han, D., Oh, S. J., Chun, S., Choe, J., Yoo, Y. (2019). CutMix: Regularization strategy to train strong classifiers with localizable features. *ICCV*.
- [46] Cubuk, E. D., Zoph, B., Shlens, J., Le, Q. V. (2020). RandAugment: Practical automated data augmentation with a reduced search space. *CVPRW*.
- [47] Hendrycks, D., Mu, N., Cubuk, E. D., Zoph, B., Gilmer, J., Lakshminarayanan, B. (2020). AugMix: A simple data processing method to improve robustness and uncertainty. *ICLR*.
- [48] Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. *CVPR*.
- [49] Riba, E., Mishkin, D., Ponsa, D., Rublee, E., Bradski, G. (2019). Kornia: An open source differentiable computer vision library for PyTorch. *WACV*.
- [50] Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *NeurIPS*.

A Appendix A: Full Hyperparameter Tables

See Table 2 in Section 6 for the complete listing of every hyperparameter swept or fixed across all five experiments.

B Appendix B: Per-Class Precision, Recall, and F1

Tables 10–12 report per-class scores across all 8 representative models on the test set. Models are ordered by tier (T1 → T4).

Table 10: Per-class F1 across all 8 models (test set).

Model	AnnualCrop	Forest	HerbVeg	Highway	Industrial	Pasture	PermCrop	Resid.	River	SeaLake
LR+PCA(200)	0.434	0.625	0.421	0.236	0.725	0.307	0.225	0.428	0.417	0.425
SVM-RBF+PCA(500)	0.643	0.878	0.604	0.439	0.849	0.750	0.533	0.674	0.623	0.834
LR+HOG	0.771	0.958	0.616	0.653	0.886	0.778	0.577	0.822	0.720	0.972
SVM-RBF+HOG	0.815	0.963	0.720	0.743	0.898	0.805	0.696	0.872	0.785	0.973
ShallowCNN (no reg)	0.950	0.980	0.923	0.949	0.964	0.960	0.915	0.985	0.935	0.983
ShallowCNN (+BN)	0.951	0.991	0.964	0.955	0.971	0.973	0.939	0.981	0.953	0.987
ShallowCNN (+BN+Drop+L2+Aug+ES)	0.908	0.965	0.884	0.923	0.938	0.908	0.859	0.953	0.894	0.983
ResNet-18	0.969	0.993	0.973	0.992	0.987	0.964	0.953	0.993	0.992	0.999

Table 11: Per-class precision across all 8 models.

Model	AnnualCrop	Forest	HerbVeg	Highway	Industrial	Pasture	PermCrop	Resid.	River	SeaLake
LR+PCA(200)	0.397	0.538	0.409	0.305	0.727	0.330	0.305	0.402	0.438	0.418
SVM-RBF+PCA(500)	0.695	0.841	0.584	0.440	0.856	0.694	0.576	0.636	0.641	0.872
LR+HOG	0.783	0.948	0.606	0.649	0.922	0.797	0.581	0.784	0.726	0.982
SVM-RBF+HOG	0.811	0.962	0.711	0.717	0.942	0.806	0.708	0.857	0.791	0.984
ShallowCNN (no reg)	0.953	0.972	0.922	0.954	0.941	0.960	0.908	0.995	0.948	0.991
ShallowCNN (+BN)	0.953	0.996	0.975	0.943	0.956	0.973	0.936	0.991	0.959	0.980
ShallowCNN (+BN+Drop+L2+Aug+ES)	0.926	0.939	0.840	0.952	0.966	0.910	0.891	0.913	0.938	0.980
ResNet-18	0.980	0.993	0.973	0.992	0.989	0.943	0.957	0.989	0.995	1.000

C Appendix C: Full Experiment B Run-by-Run Results (54 rows)

Each row is a single training run of the ShallowCNN baseline (no BN, no Dropout, no augmentation, no weight decay) for 100 epochs without early stopping. Conv is the first epoch where validation accuracy reached 0.90; --- indicates the run never reached 0.90.

Table 13: Experiment B: 6 optimizers $\times$ 3 LR $\times$ 3 seeds = 54 runs.

Optimizer	LR	Seed	Test Acc	Macro-F1	Conv@90	Best Ep	Time (s)
Adagrad	$1e-04$	42	0.6400	0.6254	—	100	98.4
Adagrad	$1e-04$	123	0.6435	0.6265	—	100	98.2
Adagrad	$1e-04$	456	0.6281	0.6105	—	100	97.8
Adagrad	$1e-03$	42	0.8091	0.7990	—	97	97.9
Adagrad	$1e-03$	123	0.8079	0.7993	—	100	98.0
Adagrad	$1e-03$	456	0.8047	0.7941	—	100	98.9
Adagrad	$1e-02$	42	0.9464	0.9439	25	97	98.5
Adagrad	$1e-02$	123	0.9499	0.9477	18	87	99.1
Adagrad	$1e-02$	456	0.9452	0.9431	20	88	100.0
Adam	$1e-04$	42	0.9138	0.9102	80	90	99.5
Adam	$1e-04$	123	0.9121	0.9088	74	98	99.8
Adam	$1e-04$	456	0.9180	0.9150	70	100	99.7
Adam	$1e-03$	42	0.9553	0.9545	10	30	101.8
Adam	$1e-03$	123	0.9506	0.9494	10	29	100.8
Adam	$1e-03$	456	0.9590	0.9581	10	34	100.3
Adam	$1e-02$	42	0.8040	0.7936	—	65	100.8
Adam	$1e-02$	123	0.8138	0.8090	—	17	100.3
Adam	$1e-02$	456	0.1111	0.0200	—	15	100.8
NAG	$1e-04$	42	0.7568	0.7416	—	100	97.7
NAG	$1e-04$	123	0.7556	0.7465	—	100	98.1
NAG	$1e-04$	456	0.7504	0.7368	—	100	98.0
NAG	$1e-03$	42	0.9205	0.9178	67	87	97.9
NAG	$1e-03$	123	0.9200	0.9175	70	90	97.8
NAG	$1e-03$	456	0.9237	0.9201	77	97	97.6
NAG	$1e-02$	42	0.9491	0.9477	12	41	98.2
NAG	$1e-02$	123	0.9533	0.9517	13	37	97.8
NAG	$1e-02$	456	0.9615	0.9607	15	50	97.4

Table 13 – continued

Optimizer	LR	Seed	Test Acc	Macro-F1	Conv@90	Best Ep	Time (s)
RMSProp	$1e-04$	42	0.9111	0.9072	80	91	98.2
RMSProp	$1e-04$	123	0.9158	0.9124	80	96	98.1
RMSProp	$1e-04$	456	0.9143	0.9110	83	98	98.4
RMSProp	$1e-03$	42	0.9499	0.9484	17	29	99.8
RMSProp	$1e-03$	123	0.9560	0.9548	14	38	99.9
RMSProp	$1e-03$	456	0.9499	0.9483	11	20	99.4
RMSProp	$1e-02$	42	0.7980	0.7909	—	17	99.3
RMSProp	$1e-02$	123	0.7763	0.7699	—	14	99.0
RMSProp	$1e-02$	456	0.7595	0.7439	—	12	98.9
SGD	$1e-04$	42	0.2435	0.1240	—	100	96.3
SGD	$1e-04$	123	0.2709	0.1704	—	100	95.9
SGD	$1e-04$	456	0.2410	0.1200	—	100	96.5
SGD	$1e-03$	42	0.7501	0.7335	—	97	96.5
SGD	$1e-03$	123	0.7489	0.7416	—	100	95.9
SGD	$1e-03$	456	0.7358	0.7262	—	98	95.7
SGD	$1e-02$	42	0.9158	0.9129	77	100	95.9
SGD	$1e-02$	123	0.9188	0.9145	80	96	96.4
SGD	$1e-02$	456	0.9148	0.9119	80	93	96.4
SGD_Momentum	$1e-04$	42	0.7575	0.7444	—	99	97.2
SGD_Momentum	$1e-04$	123	0.7558	0.7439	—	100	97.0
SGD_Momentum	$1e-04$	456	0.7484	0.7346	—	100	96.9
SGD_Momentum	$1e-03$	42	0.9222	0.9191	70	99	96.3
SGD_Momentum	$1e-03$	123	0.9277	0.9250	71	98	97.3
SGD_Momentum	$1e-03$	456	0.9225	0.9196	72	100	97.3
SGD_Momentum	$1e-02$	42	0.9477	0.9465	15	30	96.8
SGD_Momentum	$1e-02$	123	0.9607	0.9598	14	56	97.5
SGD_Momentum	$1e-02$	456	0.9553	0.9536	13	34	98.1

D Appendix D: Full Experiment C Run-by-Run Results (27 rows)

Each row is a single training run of the ShallowCNN with the indicated regularization configuration, Adam $lr=10^{-3}$.

Table 14: Experiment C: 9 configurations $\times$ 3 seeds = 27 runs.

Config	Seed	Test Acc	Macro-F1	Train Acc	Gap	Epochs	Time (s)
1_Baseline	42	0.9553	0.9545	0.9835	+0.0282	100	102.1
1_Baseline	123	0.9506	0.9494	0.9860	+0.0354	100	101.3
1_Baseline	456	0.9590	0.9581	0.9999	+0.0409	100	101.0
2a.L2_1e-4	42	0.9630	0.9621	0.9898	+0.0268	100	102.0
2a.L2_1e-4	123	0.9598	0.9585	0.9929	+0.0331	100	101.2
2a.L2_1e-4	456	0.9610	0.9600	0.9942	+0.0332	100	101.4
2b.L2_1e-3	42	0.9533	0.9521	0.9666	+0.0133	100	102.1
2b.L2_1e-3	123	0.9494	0.9476	0.9663	+0.0169	100	101.8
2b.L2_1e-3	456	0.9598	0.9584	0.9636	+0.0038	100	101.8
3a.Dropout_0.3	42	0.9501	0.9488	0.9434	-0.0067	100	110.9
3a.Dropout_0.3	123	0.9536	0.9524	0.9412	-0.0124	100	112.1
3a.Dropout_0.3	456	0.9543	0.9531	0.9403	-0.0140	100	112.2
3b.Dropout_0.5	42	0.8825	0.8788	0.8489	-0.0335	100	112.4

Table 14 – continued

Config	Seed	Test Acc	Macro-F1	Train Acc	Gap	Epochs	Time (s)
3b.Dropout.0.5	123	0.8583	0.8566	0.8551	-0.0031	100	112.4
3b.Dropout.0.5	456	0.8600	0.8592	0.8508	-0.0092	100	112.3
4.BatchNorm	42	0.9672	0.9665	0.9963	+0.0291	100	123.8
4.BatchNorm	123	0.9659	0.9650	0.9952	+0.0293	100	123.2
4.BatchNorm	456	0.9640	0.9629	0.9899	+0.0260	100	124.9
5.DataAug	42	0.9657	0.9651	0.9869	+0.0213	100	337.1
5.DataAug	123	0.9662	0.9659	0.9894	+0.0232	100	335.3
5.DataAug	456	0.9605	0.9599	0.9852	+0.0247	100	335.3
6.EarlyStop	42	0.9553	0.9545	0.9831	+0.0278	40	40.4
6.EarlyStop	123	0.9506	0.9494	0.9858	+0.0351	39	39.1
6.EarlyStop	456	0.9590	0.9581	0.9922	+0.0332	44	43.9
7.AllCombined	42	0.9240	0.9217	0.8632	-0.0608	53	196.6
7.AllCombined	123	0.9370	0.9364	0.8850	-0.0521	89	330.6
7.AllCombined	456	0.9435	0.9422	0.8894	-0.0541	95	350.9

E Appendix E: NeurIPS-Style Reproducibility Checklist

- **Datasets used.** EuroSAT RGB (Helber et al. [1]), retrieved from Zenodo doi :10.5281/zenodo.7711810. License: CC-BY-4.0.
- **Train/val/test splits.** Stratified seed-42 70/15/15 split using `sklearn.model_selection.train_test_split`. Test set used exactly once per model.
- **Source of randomness controlled.** All of `random`, `numpy`, `torch` (CPU + CUDA) seeded; `cuda.deterministic=True` `cuda.benchmark=False`. Multi-seed runs use {42, 123, 456}.
- **Model description.** Full architectures specified in Section 4; code at `src/models/`.
- **Hyperparameter selection.** Classical: 5-fold stratified CV on train+val. Deep: validation-loss-based best-epoch selection within fixed epoch budgets.
- **Range of hyperparameters considered.** Listed in Table 2.
- **Compute resources.** Single NVIDIA A100-SXM4-80GB on Google Colab Pro+. Total session: ~5 hours.
- **Number of runs.** Exp B: 54 runs (3 seeds $\times$ 6 optimizers $\times$ 3 LR). Exp C: 27 runs (3 seeds $\times$ 9 configs). Tier 4: 3 runs (3 seeds). Per-model classical: 1 CV-selected run.
- **Variance reporting.** Mean $\pm$ std over 3 seeds for all CNN tiers; CV cross-validated mean for classical.
- **Statistical significance.** Paired Student’s *t*-test on 3-seed accuracy triples for any two CNN models within 2pp (Table 8).
- **Library versions.** PyTorch 2.10, CUDA 12.8, scikit-learn 1.5+, cuML 26.02 (RAPIDS), Kornia 0.8, torchvision 0.25, OpenCV 4.x.
- **All artifacts publicly released.** <https://github.com/yashashav-dk/cmpe257-eurosat>
- **Code license.** MIT (in repository).
- **Data download.** `python data/download_eurosat.py` (script in repository).
- **Reproduction instructions.** README.md in repository root.

Table 12: Per-class recall across all 8 models.

Model	AnnualCrop	Forest	HerbVeg	Highway	Industrial	Pasture	PermCrop	Resid.	River	SeaLake
LR+PCA(200)	0.478	0.747	0.433	0.192	0.723	0.287	0.179	0.458	0.397	0.431
SVM-RBF+PCA(500)	0.598	0.918	0.624	0.437	0.843	0.817	0.496	0.716	0.605	0.800
LR+HOG	0.760	0.969	0.627	0.656	0.853	0.760	0.573	0.864	0.715	0.962
SVM-RBF+HOG	0.820	0.964	0.729	0.771	0.859	0.803	0.685	0.889	0.779	0.962
ShallowCNN (no reg)	0.947	0.989	0.924	0.944	0.987	0.960	0.923	0.976	0.923	0.976
ShallowCNN (+BN)	0.949	0.987	0.953	0.968	0.987	0.973	0.941	0.971	0.947	0.993
ShallowCNN (+BN+Drop+L2+Aug+ES)	0.891	0.993	0.933	0.896	0.912	0.907	0.829	0.998	0.853	0.987
ResNet-18	0.958	0.993	0.973	0.992	0.984	0.987	0.949	0.998	0.989	0.998

F Appendix F: Confusion Matrices

The confusion matrices for the Tier 4 ResNet-18 (mean over 3 seeds), the Tier 2 SVM-RBF + HOG, and the Tier 2 LR + HOG are shown in Figures 7–9. Confusion matrices for Tier 1 LR and Tier 1 SVM-RBF are similar in shape but with much higher off-diagonal mass on the crop-family classes; we omit them for space.

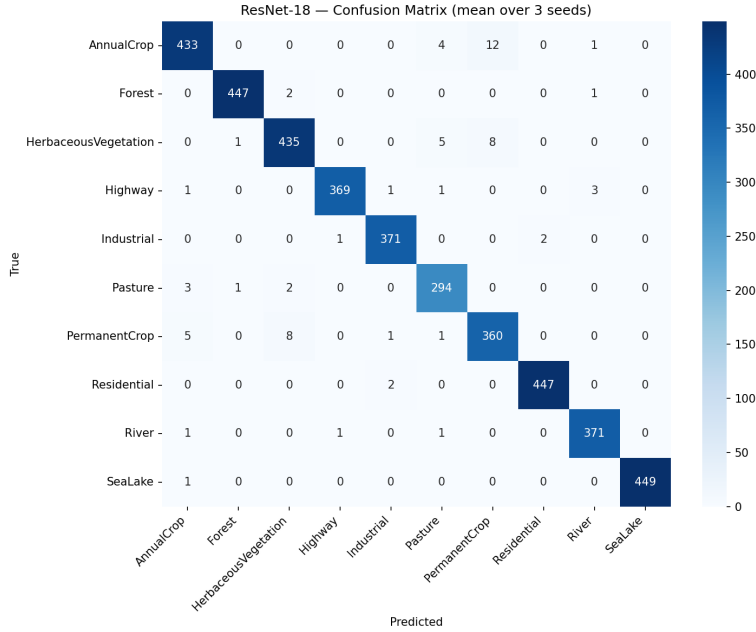


Figure 7: Tier 4 ResNet-18, confusion matrix averaged over 3 seeds.

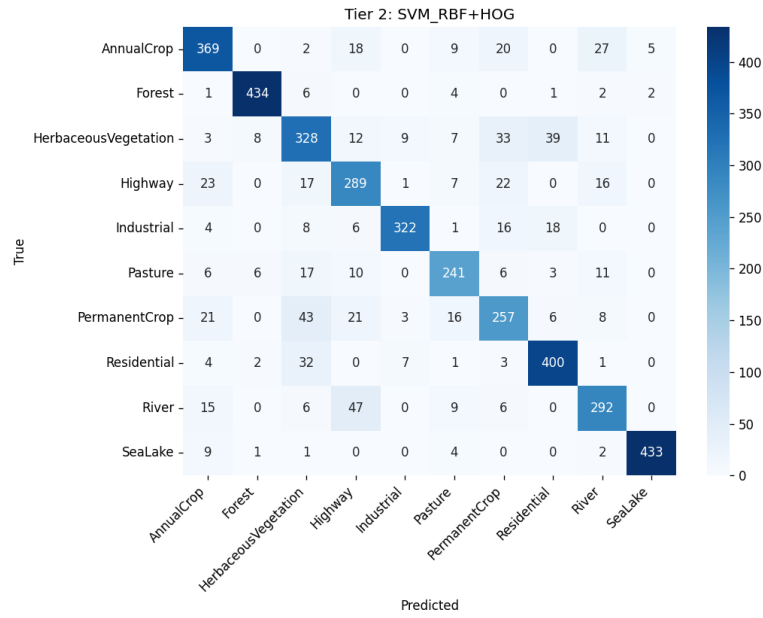


Figure 8: Tier 2 SVM-RBF + HOG, confusion matrix on test set.

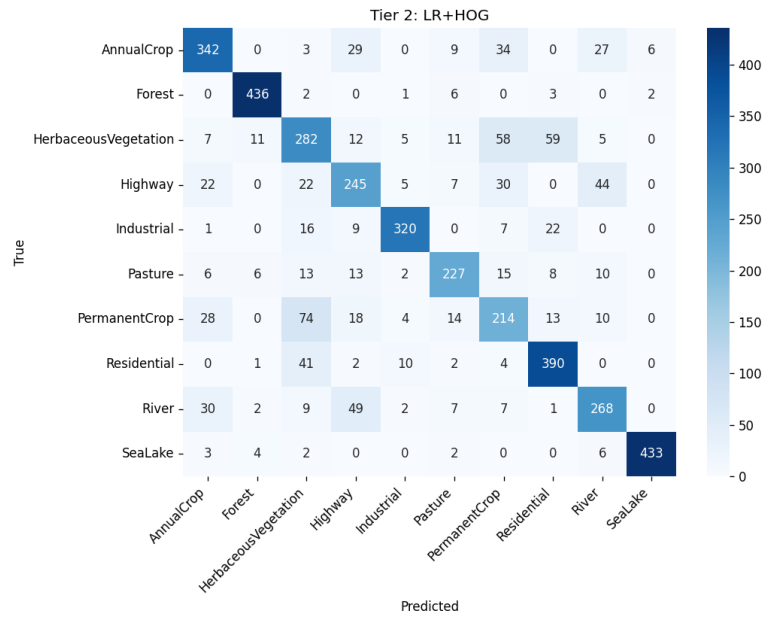


Figure 9: Tier 2 LR + HOG, confusion matrix on test set.